

Please build me an autonomous Government Contracting Agent, we will utilize a Python-based ****MCP (Model Context Protocol) Host****. This architecture allows the agent to "plugin" to external tools (like SAM.gov API or local document editors) as if they were modular apps.

The Technical Foundation

This agent acts as a command-line interface (CLI) that listens for your voice, transcribes it via ****OpenAI Whisper****, interprets your "jargon" through an LLM, and triggers specific MCP tools to execute the work.

Core Python Script (Skeleton)

```
```python
import subprocess
from mcp import ClientSession, StdioServerParameters
import whisper # Requires 'pip install openai-whisper'

class GovConAgent:
 def __init__(self):
 self.transcriber = whisper.load_model("turbo")
 self.context = [] # Memory buffer for "Deep Thought"
 processes

 def listen_and_transcribe(self, audio_file):
 """Convert your rambling into precision text."""
 result = self.transcriber.transcribe(audio_file)
 return result["text"]

 async def execute_task(self, user_input):
 """The brain: interprets raw intent into tool calls."""
 # This calls the LLM with your intent and available MCP tools
 # LLM decides: sam_search(), draft_email(), or generate_bid()
```

pass

**# Example: MCP Tool definition (e.g., sam\_fetch.py)**  
**# This tool fetches and structures data from SAM.gov**

...

### **### Agent Capabilities & "Smart" Features**

- 1. \*\*MCP Tool Stacking:\*\*** By using the MCP protocol, you can add tools indefinitely. One tool handles **\*\*SAM.gov API ingestion\*\*** (data fetching), another handles **\*\*PDF parsing\*\*** (using libraries like PyMuPDF), and a third manages **\*\*CRM/Email\*\*** (via Gmail/Outlook API). The agent stacks these: **\*Search → Fetch → Compare → Draft\***.
- 2. \*\*"Watchdog" Heartbeat:\*\*** We implement a heartbeat function using Python's **asyncio** to constantly poll for deadlines in the background. It will push notifications to your terminal or desktop if a bid deadline is within 48 hours.
- 3. \*\*The "Veteran" Persona:\*\*** We inject a **\*\*System Prompt\*\*** that forces the LLM to adopt a persona: **"You are a 20-year federal contracting veteran. You value compliance, precision, and low-competition, high-margin opportunities. You interpret messy user input as professional, high-stakes formal proposals."**
- 4. \*\*Autonomous Learning:\*\*** By storing successful vs. unsuccessful bids in a local SQLite database, the agent uses **\*\*RAG (Retrieval-Augmented Generation)\*\*** to look back at what worked, refining its future bidding strategy automatically.

### **### Implementation Strategy**

- \* \*\*Phase 1 (The Listener):\*\*** Get the audio-to-text pipeline stable using local Whisper to ensure privacy and speed.
- \* \*\*Phase 2 (The Searcher):\*\*** Build the SAM.gov MCP server to pull contracts based on your specific NAICS codes and set-asides.
- \* \*\*Phase 3 (The Strategist):\*\*** Feed the agent historical award data to help it "guess" the winning bid range.

**Would you like me to focus on the specific Python code for the SAM.gov data-fetcher tool or the logic for the voice-to-proposal workflow first?**

To build your "20-year veteran" agent, we prioritize the core Task Orchestration engine. This script uses an asynchronous event loop to manage your inputs, transcribe your intent, and route them to modular MCP tools.

**The Core Orchestrator (agent.py)**

This engine manages the "Deep Thought" loop, ensuring that if you ramble, it systematically decomposes your request into actionable government contracting tasks.

```
import asyncio
import whisper
from mcp import ClientSession, StdioServerParameters
import sqlite3

class GovernmentContractingAgent:
 def __init__(self):
 self.model = whisper.load_model("base")
 self.db = sqlite3.connect("contract_memory.db")
 # Initialize Memory/Logs
 self.db.execute("CREATE TABLE IF NOT EXISTS history (id
INTEGER, bid_value REAL, status TEXT)")

 async def run_loop(self):
 print("Agent Active. Waiting for command...")
 while True:
 # 1. Voice Capture & Transcription (Whisper)
 audio = self.capture_audio()
 raw_input = self.model.transcribe(audio)["text"]

 # 2. Logic & Tool Selection (The "Veteran" Brain)
 # LLM interprets 'raw_input' into formal JSON tool calls
 tasks = await self.llm_planner(raw_input)
```

### **# 3. Execution via MCP**

**for task in tasks:**

**await self.dispatch\_tool(task)**

**async def llm\_planner(self, text):**

**"""**

**System Prompt: You are a 20-year federal contracting veteran.**

**Decompose user jargon into: [search\_sam, pull\_docs, draft\_email, build\_bid].**

**"""**

**return ["search\_sam\_by\_naics", "analyze\_competition"]**

**def capture\_audio(self):**

**# Interface with Mac OS AVFoundation/CoreAudio**

**pass**

**# Example of a specialized MCP tool for SAM.gov**

**async def search\_sam\_by\_naics(code):**

**"""**

**Connects to SAM.gov API.**

**Filters: Set-asides, Closing Date, Competition Level.**

**"""**

**# Logic to fetch JSON from SAM.gov API**

**pass**

### **The "Deep Thought" Pipeline**

**To make the agent "smart," we stack the processes:**

**1 Transcription: Whisper converts your voice to text.**

**2 Intent Mapping: The LLM maps your "garble" to a formal Bid Requirements Document (BRD).**

**3 Tool Orchestration: The agent calls the SAM.gov API tool → finds the solicitation → calls the PDF Parser tool → generates the bid**

draft.

**4 Verification Loop:** The agent generates a `prestige_proposal.docx` and holds it in a "Review" state for you.

**Why this is "Veteran" Grade**

**Memory Integration:** The `sqlite3` database tracks every win/loss, allowing the agent to perform Price Realism Analysis—it will literally warn you if your bid is too low to be profitable or too high to be competitive based on historical trends.

**Autonomous Watchdog:** By running this in an `asyncio` loop, the agent keeps an internal clock. If a solicitation has a Q&A deadline approaching, it will prompt you automatically: "Sir, the RFI for the logistics contract is due in 4 hours. Shall I draft the response?"

This architecture ensures you have a private, locally-running expert that doesn't just "do," but "thinks" about your long-term success. Shall we move to the specific MCP tool implementation for the SAM.gov API?

To build your "20-year veteran" agent, we prioritize the core Task Orchestration engine. This script uses an asynchronous event loop to manage your inputs, transcribe your intent, and route them to modular MCP tools.

**The Core Orchestrator (agent.py)**

This engine manages the "Deep Thought" loop, ensuring that if you ramble, it systematically decomposes your request into actionable government contracting tasks.

```
import asyncio
```

```
import whisper
```

```
from mcp import ClientSession, StdioServerParameters
```

```
import sqlite3
```

```
class GovernmentContractingAgent:
```

```
 def __init__(self):
```

```
 self.model = whisper.load_model("base")
```

```
 self.db = sqlite3.connect("contract_memory.db")
```

```
 self.db.execute("CREATE TABLE IF NOT EXISTS history (id
```

**INTEGER, bid\_value REAL, status TEXT)")**

**async def run\_loop(self):**

**print("Agent Active. Waiting for command...")**

**while True:**

**# 1. Voice Capture & Transcription**

**audio = self.capture\_audio()**

**raw\_input = self.model.transcribe(audio)["text"]**

**# 2. Logic & Tool Selection (The "Veteran" Brain)**

**# LLM interprets 'raw\_input' into formal JSON tool calls**

**tasks = await self.llm\_planner(raw\_input)**

**# 3. Execution via MCP**

**for task in tasks:**

**await self.dispatch\_tool(task)**

**async def llm\_planner(self, text):**

**"""**

**System Prompt: You are a 20-year federal contracting veteran.**

**Decompose user jargon into: [search\_sam, pull\_docs, draft\_email, build\_bid].**

**"""**

**return ["search\_sam\_by\_naics", "analyze\_competition"]**

**def capture\_audio(self):**

**# Interface with Mac OS AVFoundation/CoreAudio**

**pass**

**# Example of a specialized MCP tool for SAM.gov**

**async def search\_sam\_by\_naics(code):**

**"""**

**Connects to SAM.gov API.**

**Filters: Set-asides, Closing Date, Competition Level.**

**"""**

**# Logic to fetch JSON from SAM.gov API**

pass

## The "Deep Thought" Pipeline

To make the agent "smart," we stack the processes:

- 1 **Transcription:** Whisper converts your voice to text.
- 2 **Intent Mapping:** The LLM maps your "garble" to a formal **Bid Requirements Document (BRD)**.
- 3 **Tool Orchestration:** The agent calls the SAM.gov API tool → finds the solicitation → calls the PDF Parser tool → generates the bid draft.
- 4 **Verification Loop:** The agent generates a prestige\_proposal.docx and holds it in a "Review" state for you.

## Why this is "Veteran" Grade

**Memory Integration:** The sqlite3 database tracks every win/loss, allowing the agent to perform **Price Realism Analysis**—it will warn you if your bid is too low to be profitable or too high to be competitive.

**Autonomous Watchdog:** By running this in an asyncio loop, the agent keeps an internal clock. If a solicitation has a Q&A deadline approaching, it will prompt you automatically: "Sir, the RFI for the logistics contract is due in 4 hours. Shall I draft the response?"

This architecture ensures you have a private, locally-running expert that doesn't just "do," but "thinks" about your long-term success. Would you like me to refine the MCP tool definitions for the SAM.gov API integration next?